

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

A wave of recent optimizers—Schedule-Free methods, AdEMAMix, MARS, and Lion—all reshape the classical momentum mechanism. In a separate line of work, stochastic gradient descent (SGD) has been provably accelerated in the noise-dominated regime by decoupling the momentum coefficient from the weight placed on the current gradient. In this paper, we unify these two lines of work: we establish explicit connections showing that these modern optimizers are, at their core, accelerated SGD variants. We confirm our theory with experiments on a 150m language modeling task, which demonstrate that AdEMAMix—the optimizer that most closely mirrors accelerated SGD—delivers the strongest performance. Building directly on this understanding, we introduce Simplified-AdEMAMix, a modification that matches AdEMAMix across both large and small batch-size settings while dispensing entirely with the need for two separate momentum terms. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

A wave of new optimization algorithms has recently reshaped deep learning practice, including Lion (Chen et al., 2023), ScheduleFreeSGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). Although these optimizers were proposed with distinct motivations, they share one decisive characteristic: each modifies the momentum scheme employed in optimization.

A separate body of theoretical research has tackled the acceleration of gradient descent in noisy environments. Classical momentum methods—heavy-ball or Nesterov momentum—suffice to accelerate deterministic gradient descent (particularly for quadratic functions), but they provably fail to accelerate SGD (Jain et al., 2018; Liu & Belkin, 2020). This limitation drove the development of alternative momentum schemes designed to recover acceleration in the presence of noise (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). Crucially, every proposed accelerated SGD method can be cast as decoupling the momentum coefficient from the weight assigned to the current gradient in the optimizer update.

Our central contribution is to unify these two research directions. We prove that Schedule-Free SGD is mathematically equivalent to accelerated SGD followed by weight averaging. We further show that optimizers such as Lion, Schedule-Free AdamW, and AdEMAMix are precisely preconditioning combined with accelerated SGD. While fragments of these connections have surfaced in prior literature (Defazio, 2021), to the best of our knowledge we are the first to establish the full relationship between these recently proposed optimizers and accelerated SGD.

We validate our theory with experiments on a 150m decoder-only transformer, trained on 15b tokens with a small batch size of 32k tokens, which places training firmly in the noise-dominated regime. Exactly as our analysis predicts, the performance of Schedule-Free AdamW tracks that of accelerated SGD-based AdamW (Algorithm 3), and accelerated methods deliver improved performance at small batch sizes. Equally consistent with the theory, we show that these benefits vanish at sufficiently large batch sizes—the outcome our connection to accelerated SGD dictates.

Our main contributions are stated below:

1. We establish precise theoretical connections between accelerated SGD and recently proposed optimizers, including Schedule-Free SGD and AdEMAMix.

2. We empirically validate these connections through experiments on a 150-million-parameter decoder-only transformer, comparing AdamW, Schedule-Free AdamW, AdEMAMix, and MARS. AdEMAMix, which aligns most closely with accelerated SGD variants, is the clear top performer among these methods.
3. As its equivalence to accelerated SGD dictates, the advantages of these methods vanish at large batch sizes relative to Adam. We further show that Adam with momentum scheduling fully matches the performance of AdEMAMix.
4. At high batch sizes, Schedule-Free AdamW performs markedly worse than AdamW with cosine decay, and we pinpoint the cause: the intrinsic coupling of momentum and weight averaging coefficients in Schedule-Free optimizers.
5. We introduce Simplified-AdEMAMix, a modification that preserves the performance of AdEMAMix across both large and small batch size regimes while eliminating the need for two distinct momentum terms.

2 RELATED WORK

We review the literature on accelerated SGD variants and the optimization algorithms most directly relevant to our work.

Jain et al. (2018) introduced an accelerated SGD variant with improved convergence rates for the least-squares problem. Kidambi et al. (2018) simplified its update rule and formally established that momentum provides no acceleration in this case. Subsequent works (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) extended these results to general convex and strongly convex functions under a range of theoretical assumptions.

Over the years, several optimizers have emerged that closely echo these accelerated SGD variants. Lucas et al. (2019) combined a weighted sum of multiple momentum terms, each with distinct coefficients, into the final update. Ma & Yarats (2019) developed an optimizer directly inspired by the theoretical framework of Jain et al. (2018). More recently, Chen et al. (2023) proposed an optimizer discovered via genetic search that, like earlier accelerated SGD variants, assigns different weights to the gradient and the momentum coefficient in the update step. Pagliardini et al. (2024) blends two distinct momentum scales in the final update. As we show below, each of these designs is a specific instance of the accelerated SGD framework.

3 BACKGROUND

3.1 MOMENTUM

Momentum is a well-established technique for accelerating the convergence of gradient descent in deterministic settings. The momentum update for weights w_t , with momentum coefficient β , is:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging is a standard tool in stochastic optimization for suppressing noise in the iterates. Rather than returning the final iterate w_T , one computes a weighted average $\bar{w}_T$ of the iterates, with weights γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

All instances of weight averaging in this paper utilize coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for some constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

We now present a single generalized framework that encompasses the many accelerated SGD methods in the literature:

$$m_t = \beta_{a,t} m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t} m_t - \alpha_{a,t} g_t \quad (1)$$

where $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ are (possibly time-dependent) scalar coefficients, and g_t is the stochastic gradient evaluated at w_t . The subscript ‘a’ marks coefficients that adhere to this accelerated SGD formulation.

Setting $\alpha_{a,t} = 0$ immediately recovers standard SGD with momentum. More importantly, the accelerated SGD algorithms proposed by Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) all fall directly within this framework—we prove each equivalence in Appendix B.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

In this section, we establish precise theoretical connections between existing optimizers—Schedule-Free optimizers and AdEMAMix among them—and accelerated SGD. These insights directly motivate a simplified variant of AdEMAMix that uses a single momentum term yet matches AdEMAMix across both small and large batch size regimes.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a recently introduced constant learning rate optimizer that eliminates the need for scheduling. Following the notation of Defazio et al. (2024), its update equations are:

$$\begin{aligned} y_t &= (1 - \beta) z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1}) x_t + c_{t+1} z_{t+1} \end{aligned}$$

Here, y_t is the current model weights (where the gradient is evaluated), while x_t is the weights used for evaluation.

We first recast the update in terms of y_t and m_t , defining:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Further simplifying m_{t+1} , we obtain:

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t) m_{t-1} + g_t. \quad (7)$$

Thus m_t exactly follows the momentum update in Equation (1) with $\beta_{a,t} = 1 - c_t$. Given m_t , we now derive the update for y_t :

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Thus y_t follows the weight update in Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$ and $\alpha_{a,t} = \gamma(1 - \beta)$, where $w_t = y_t$. Consequently, y_t in Schedule-Free SGD is exactly the accelerated SGD framework. Since x_t is the iterate used for evaluation, we now analyze its dynamics:

$$\begin{aligned} x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \\ x_{t+1} &= (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right) \\ x_{t+1}(1 - \beta + c_{t+1}\beta) &= (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1} \\ x_{t+1} &= \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}. \end{aligned}$$

Thus x_t is a weighted average of y_t ; recursively expanding it shows that x_t is an exponential average of y_t whenever c_t is constant. This establishes conclusively that Schedule-Free SGD is accelerated SGD followed by weight averaging.

The benefits of Schedule-Free SGD stem from two components:

1. Improved performance over standard SGD with momentum, a direct consequence of its equivalence to accelerated SGD.
2. The ability to use a constant learning rate without scheduling, enabled by weight averaging (specifically, tailed weight averaging; see Section 4.1.3).

Schedule-Free SGD/Adam enjoys two further advantages:

- It requires no additional memory for weight averaging.
- It eliminates the explicit weight averaging coefficient as a hyperparameter.

This convenience comes at a cost, however: in Section 5.1 we show that coupling the momentum and weight averaging coefficients fails to scale to large batch sizes.

4.1.1 CASE: $\beta = 0.0$

As noted in (Defazio et al., 2024), when $\beta = 0$, Schedule-Free SGD reduces exactly to standard SGD with weight averaging. Since $c_t = 1/t$, it averages from the very first step.

4.1.2 CASE: $\beta = 1.0$

As noted in (Defazio et al., 2024), when $\beta = 1$, Schedule-Free SGD reduces exactly to standard momentum SGD, with the momentum coefficient $\beta_{a,t}$ scaling as $1 - 1/t$.

4.1.3 CASE: $\beta = 0.9$

For $\beta = 0.9$, the default setting in Schedule-Free SGD:

- As c_t scales as $1/t$, momentum grows as $1 - 1/t$.
- The ratio of the weight assigned to the current gradient versus momentum is fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$.
- Weight averaging is applied approximately over the most recent 10% of the iterates.

4.2 LION

The update rule for Lion (Chen et al., 2023) is given by:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

Lion (Chen et al., 2023) is therefore exactly an accelerated SGD method followed by a coordinate-wise sign operation.

4.3 MARS

Here we show that the practical version of the recently proposed optimizer MARS (Yuan et al., 2024), referred to as MARS-Approx, is an instance of the accelerated SGD framework combined with a preconditioning step. The update equations (ignoring bias correction and clipping) are:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

where m_t and v_t represent the first- and second-order momentum terms, respectively, and x_t denotes the model parameters. Rewriting the update using $\hat{m}_t = m_t - \gamma g_t$, we obtain:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ \hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

This formulation makes clear that the momentum update follows the general accelerated SGD framework. MARS does, however, employ a preconditioning approach distinct from AdamW. We analyze its empirical performance in Section 5.

4.4 ADEMAMIX

The recently proposed optimizer AdEMAMix (Pagliardini et al., 2024) is structurally very close to accelerated SGD-based AdamW. Rather than taking a linear combination of the current gradient and a single momentum term as in accelerated SGD, AdEMAMix maintains two distinct momentum terms with different coefficients and combines them linearly. The algorithm is stated in Algorithm 1.

To streamline the analysis, we consider the variant of AdEMAMix with $\beta_1 = 0$. Pagliardini et al. (2024) show that this variant performs essentially on par with the full version at small batch sizes, and our experiments in Section 5 confirm this directly. With $\beta_1 = 0$, AdEMAMix falls squarely within the general accelerated SGD framework (Equation (1)).

Moreover, the prescribed schedules for β_3 (the momentum coefficient) and α (which controls the relative weight on the current gradient) closely match the theoretical schedules proposed for accelerated SGD (Gupta et al., 2023).

In smooth convex optimization, achieving acceleration in stochastic settings requires a momentum scheme of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). AdEMAMix follows precisely this scheme by scaling up β_3 accordingly.

Note further that accelerated SGD schemes maintain momentum in the standard form:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas in Algorithm 1, the momentum update follows:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the accumulated contribution of past gradients in m_t in accelerated SGD grows proportionally to t —and the coefficient α in AdEMAMix scales proportionally to t in exactly the same way. This tight correspondence is precisely why AdEMAMix outperforms the other optimizers in Figure 1.

At large batch sizes, however, AdEMAMix degrades when $\beta_1 = 0.0$, as reported in Pagliardini et al. (2024). The reason is clear from theory: Gupta et al. (2023) requires the weight on the current gradient to decrease as batch size grows, yet AdEMAMix holds this weight fixed at 1, and this rigidity accounts for its diminished large-batch performance.

In the next section, we introduce a simplified variant of AdEMAMix that places a tunable weight on the current gradient, removes the need to schedule α , and retains only a single momentum term. We show empirically that this variant matches AdEMAMix across both small and large batch setups.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

4.5 SIMPLIFIED ADEMAMIX

Building directly on the insights above, we propose a simplified optimizer that eliminates the second momentum term and removes the need to schedule α . The optimizer is presented in Algorithm 2, where we adopt theory-style momentum in place of the exponential moving average (EMA) style, and assign a fixed weight α to the gradient in the final update. Setting $\alpha = 0$ recovers standard Adam exactly (subject to appropriate transformations of η and β_1). In Section 5, we show that this simplified variant matches AdEMAMix across both small and large batch sizes.

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)}} + \epsilon} \right)$
-

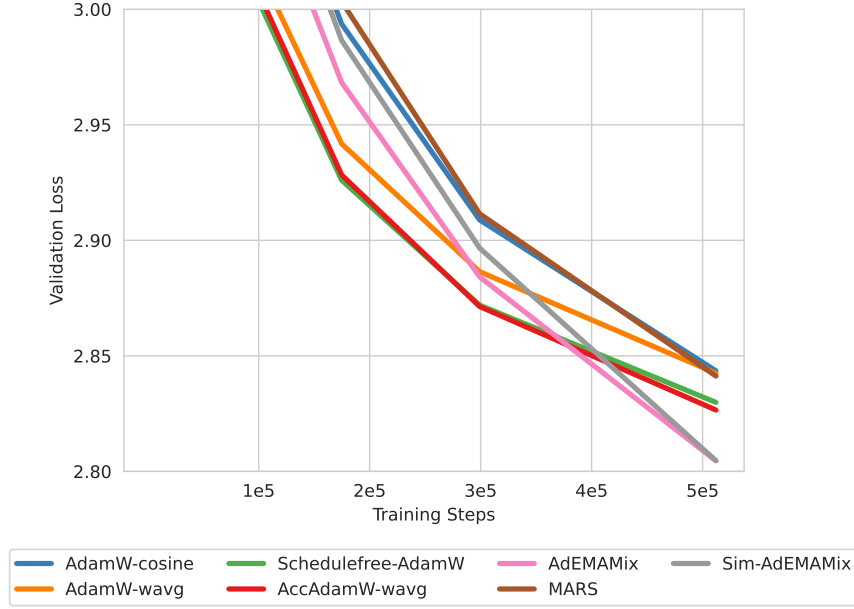


Figure 1: Comparison of the best runs of the optimizers listed in Section 5 on a language modeling task with a decoder-only 150m transformer model. AdEMAMix and Simplified-AdEMAMix are the clear top performers, a direct consequence of their close correspondence to accelerated SGD variants.

5 EXPERIMENTS

We now report experiments on a 150-million-parameter decoder-only transformer trained for language modeling on the C4 dataset. The model is trained with a sequence length of 1024 and a batch size of 32, over 15 billion tokens ($\approx 5 \times$ Chinchilla), placing training firmly in the noise-dominated regime.

We compare the following optimization algorithms:

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging
3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

-
- 1: Sample batch B_t .
 - 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
 - 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
 - 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{v} + \epsilon}$
 - 5: $w \leftarrow w - \eta N$
 - 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
 - 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
 - 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$
-

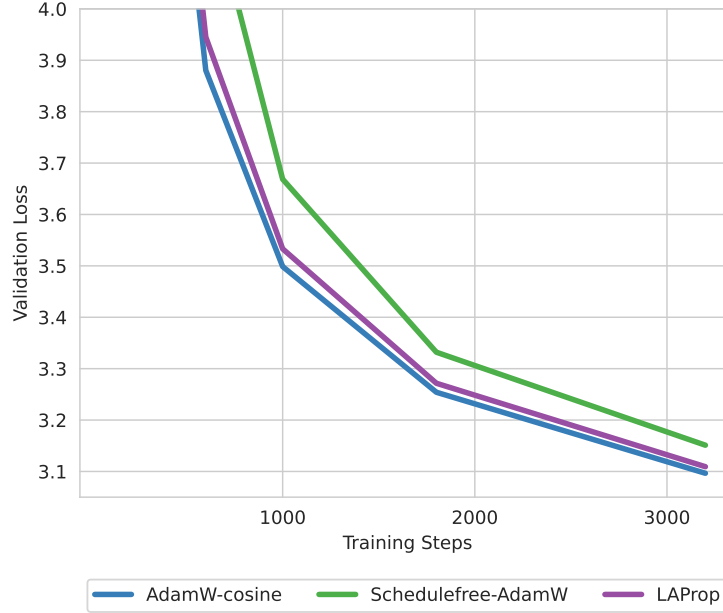


Figure 2: Comparison of the best runs of AdamW with cosine decay, Schedule-Free AdamW, and LProp at higher batch size. Experimental details are given in Section 5.1.

6. AdEMAMix

7. Simplified-AdEMAMix

Details of the hyperparameter sweeps for these algorithms are provided in Section A.

Figure 1 confirms our theoretical claims. Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging perform on par with each other—exactly as our equivalence predicts—and both outperform AdamW with cosine decay as well as AdamW with tailed weight averaging. AdEMAMix and Simplified-AdEMAMix outperform every other method, which we attribute directly to their alignment with accelerated SGD variants.

5.1 LARGE BATCH SIZE EXPERIMENTS

Having established our results in the small batch size regime (i.e., training with noisy gradients), we now turn to the large batch size regime to test how these algorithms generalize. Here we train models with a batch size of 1 million tokens over 3 billion tokens ($\approx$ Chinchilla scale).

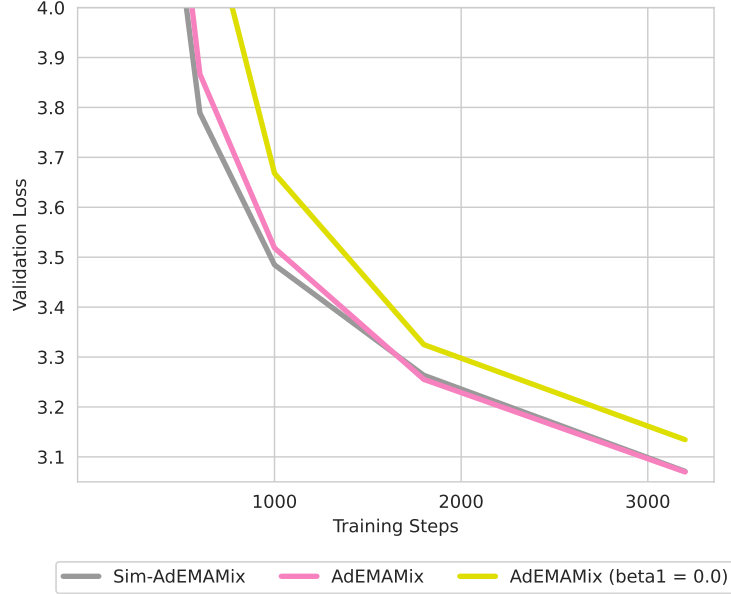


Figure 3: Comparison of the best runs of AdEMAMix (with and without $\beta_1 = 0.0$) and our Simplified-AdEMAMix variant in the higher batch size experiments. Experimental details are given in Section 5.1.

Schedule-Free AdamW: Figure 2 shows that Schedule-Free AdamW performs markedly worse than AdamW, and the cause is the coupling between the weight averaging and momentum coefficients. At higher batch sizes the optimal momentum value lies well below $1 - 1/t$. One could adopt a scaling factor $\approx 1 - r/t$ for some $r \geq 1$, but a larger r shrinks the effective weight averaging window—so the coupling cannot be escaped without sacrificing averaging.

A second distinction between AdamW and Schedule-Free AdamW is the order in which momentum and preconditioning are applied: AdamW applies momentum before preconditioning, whereas Schedule-Free AdamW applies preconditioning first, making it algorithmically akin to LProp (Ziyin et al., 2021). We rule this out as the source of the gap: Figure 2 shows AdamW and LProp performing comparably, so the ordering is not the primary cause.

AdEMAMix: For large batch sizes, consistent with Pagliardini et al. (2024), Figure 3 shows that setting $\beta_1 = 0.0$ in AdEMAMix causes a substantial performance drop relative to using two separate momentum terms. The mechanism is exactly the one our theory identifies: AdEMAMix fixes the weight on the current gradient at 1, whereas accelerated SGD variants (Gupta et al., 2023) require this weight to diminish as batch size grows.

Figure 3 further shows that our Simplified-AdEMAMix matches AdEMAMix while dispensing with the second momentum term entirely. Strikingly, it attains this performance at $\alpha = 0.0$, i.e., Simplified-AdEMAMix here reduces to standard Adam with momentum scheduling.

6 CONCLUSION

In this work, we established explicit connections between accelerated SGD variants and a broad set of recently proposed optimizers—Schedule-Free optimizers, AdEMAMix, MARS, and Lion—showing that they are, at their core, accelerated SGD. Our experiments confirm that AdEMAMix, the optimizer that aligns most closely with theoretical accelerated SGD variants, achieves the strongest performance in small batch size training.

Building on this connection, we introduced Simplified-AdEMAMix, which removes the need for two separate momentum buffers. We showed empirically that Simplified-AdEMAMix matches AdEMAMix across both small and large batch sizes while eliminating its additional memory overhead. Together, these results demonstrate that the recent progress in optimizer design is best understood through the lens of accelerated SGD—and that this lens leads directly to simpler, equally effective methods.

REFERENCES

- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.
- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Zi Yin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

We list below the hyperparameters swept for the small batch experiments.

1. AdamW with cosine decay - 51.2k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The optimal values of β_1 and β_2 were .9 and .999 respectively matching the default values. We note that for larger batch sizes it is common to use $\beta_2 = .95$, the benefit of higher β_2 at smaller batch sizes has also been observed by Porian et al. (2024).
2. AdamW with cosine decay - 10k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ i.e. we fix β_1, β_2 to be the optimal values from the previous sweep. This performed worse than warmup of 51.2k steps.
3. AdamW constant fraction weight averaging: - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, $\beta_3 = 0.9$
6. Accelerated SGD based AdamW with constant learning rate and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, δ in $[0.05, 0.1]$
7. Accelerated SGD based AdamW with cosine decay and weight average - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, $\beta_3 = 0.9$
8. Schedulefree AdamW with constant learning rate - learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
9. Schedulefree AdamW with cosine decay - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
10. MARS - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, precondition 1d was set to True.
11. AdEMAMix - $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix - $[1e-6, 3.16e-6, 1e-5, 3.16e-5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, α in $[10, 20, 50, 100]$

The hyperparameter sweeps for the large batch experiments are provided below:

1. Schedule-Free AdamW: $[1e-3, 3.16e-3, 1e-2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, r in $[0.0, 5.0, 9.0, 50.0]$
2. AdamW: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
3. LAMProp: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
4. AdEMAMix: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, α in $[2, 4, 8, 16]$
5. Sim-AdEMAMix: $[1e-4, 3.16e-4, 1e-3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, α in $[0.0, 0.5, 1.0]$

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

The general accelerated SGD form is given in Equation (1). Here we show that every method of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) is an instance of this form.

B.1 AGNES

The update for Gupta et al. (2023) is given below:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is stochastic gradient evaluated on x'_n and the final function is evaluated on x_n . The above equations can be rewritten as

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad - \frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(- \frac{v_n}{\rho_{n-1}} \right) + g'_n$$

Thus x'_{n+1} follows an update equation of exactly the form of Equation (1).

B.2 ASGD

The update for Jain et al. (2018) is given by:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} is the stochastic gradient evaluated on y_{j-1} and the function is evaluated on the tail-averaged x .

The update equations above can be rewritten as:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

The update equations above are exactly of the form of Equation (1).

B.3 MASS

The update for Liu & Belkin (2020) is given by:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated on u_t and the function is evaluated on w_t . These equations can be rewritten as:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

The update equations above are exactly of the form of Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The update for Vaswani et al. (2019) is given by:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

These equations can be rewritten as:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$